DMBI Module 6

Explain Bagging of a classifier

Bagging (Bootstrap Aggregating) is an ensemble technique that improves stability and accuracy by:

1. Creating multiple training sets by drawing *bootstrap samples* (random sampling with replacement) from the original dataset. Each sample is same size as original (~63% unique rows per sample).
2. Training a classifier (e.g., decision tree, `SVM`) on each bootstrap sample independently.
3. Aggregating predictions:
   - Classification: majority voting.
   - Regression: averaging.

Reduces variance without increasing bias. If individual models overfit different parts of the data, averaging cancels out overfitting errors.

What are ensemble methods? Why useful?

Ensemble methods combine multiple individual models (called “base learners”) to produce one optimal predictive model.

Why useful:

- Reduce variance (Bagging, Random Forest) → avoid overfitting.
- Reduce bias (Boosting) → improve underperforming models.
- Improve generalization → better performance on unseen data.
- Robustness → errors of one model are compensated by others.

Random Forest

Random Forest is an extension of Bagging applied to decision trees, with an extra layer of randomness:

- Builds many decision trees on bootstrap samples (like Bagging).
- At each node split, only a random subset of features is considered (not all features).

- Final prediction: majority vote (classification) or average (regression).

Key advantages:

- Handles high-dimensional data well.
- Provides feature importance.
- Robust to overfitting.
- Handles missing values (through proximity matrices).

Bootstrapping

Bootstrapping is a statistical resampling technique where:

- Repeatedly draw random samples with replacement from an observed dataset.
- Each bootstrap sample has the same size as the original dataset.
- Some observations appear multiple times, some not at all (on average ~63% unique).

Uses in ML:

- Estimating sampling distribution of a statistic (mean, variance).
- Bagging and Random Forest for creating diverse training sets.
- Confidence interval estimation.

Cross Validation

Cross-validation (`CV`) is a model evaluation technique that assesses how well a model generalizes to unseen data.

k-Fold CV process:

1. Split data into `k` equal folds.
2. For each fold `i` :
   - Train model on the other `k-1` folds.
   - Test model on fold `*` i*.
3. Average performance across k tests.

Common variants:

- `k=5` or `10` (standard).

- Leave-one-out `CV` (`k = N`, expensive).
- Stratified `CV` (preserves class distribution).

Purpose: Detect overfitting, tune hyperparameters, avoid train-test leakage.

Explain Confusion Matrix

A confusion matrix is a table that visualizes the performance of a classifier on test data where true labels are known.

For binary classification:

	Predicted: Positive	**Predicted: Negative**
Actual: Positive	`TP` (True Positive)	`FN` (False Negative) – Type II error
Actual: Negative	`FP` (False Positive) – Type I error	`TN` (True Negative)

Derived metrics:

- Accuracy = (`TP` + `TN`) / Total
- Precision = `TP` / (`TP` + `FP`)
- Recall (Sensitivity) = `TP` / (`TP` + `FN`)
- F1-score = 2 *(Precision* Recall) / (Precision + Recall)
- Specificity = `TN` / (`TN` + `FP`)

Design a BI System for Fraud Detection (Full Pipeline)

Step 1: Data Collection

- Sources: Transaction logs, customer profiles, geolocation, device fingerprints, historical chargebacks, external blacklists.
- Formats: Structured (`SQL` tables), semi-structured (`JSON` logs), real-time streams (`Kafka`).
  Step 2: Data Integration & Storage
- `ETL/ELT`: Ingest into a data lake (raw) + data warehouse (`processed`).
- Real-time layer: Use in-memory DB (`Redis`) or stream processing (`Apache Flink`).
- Storage: Star schema with fact table (transactions) and dimensions (customer, merchant, time, device).
  Step 3: Data Preprocessing

- Handle missing values, remove duplicates.
- Feature engineering:
  - Velocity features: transaction count per user per hour.
  - Location mismatch: distance from last login.
  - Amount anomaly: deviation from user's usual amount.
  - Time features: transaction hour, weekend flag.
- Normalization / scaling.
  Step 4: Model Development (ML Layer)
- Labeling: Historical transactions labeled "`fraud`" / "`legit`" (imbalanced).
- Model choice: Random Forest (interpretable), XGBoost, or neural network.
- Training: Use stratified `k-fold`, `SMOTE` for imbalance.
- Output: Fraud probability score (`0 to 1`).
  Step 5: Decision Engine (Rules + ML)
- Hybrid approach:
  - Rules: If `score > 0.95` → auto-block. If `< 0.3` → auto-approve.
  - Middle zone (`0.3–0.95`) → trigger manual review or second-factor auth.
- Real-time scoring via REST API.
  Step 6: Visualization & Monitoring Dashboard (BI)
- Metrics: True positive rate, false positive rate, $ amount saved / lost.
- Charts: Daily fraud rate trend, top fraud patterns, heatmap by location.
- Alerts on sudden FPR spike or model drift.
  Step 7: Feedback Loop & Decision Making
- Manual review outcomes fed back into retraining pipeline (weekly).
- Business action: freeze account, refund, block merchant.